Programming for AI
Week 4 Activity

Task 1

Write a script that maintains a global variable `call_count = 0`. Create a function `execute_task()` that increments this global counter every time it is called.

- Without passing the variable as an argument, ensure the function modifies the global instance rather than creating a local one.
- Why is this pattern generally avoided in large-scale AI pipelines?

Task 2

Write a function **`math_operation(operator, *numbers)`** where:

- `operator` is a string (e.g., `"add"`, `"mul"`).
- `*numbers` collects any amount of numerical data.
- If the operator is `"add"`, return the sum. If `"mul"`, return the product.

  Example: `math_operation("mul", 2, 3, 4)` should return `24`.

Task 3

In Machine Learning, we often pass many settings to a model. Write a function **`print_config(model_name, **hyperparams)`** that:

1. Prints the `model_name` in uppercase.
2. Iterates through the `**hyperparams` and prints them in the format: `[Setting Name] -> [Value]`.

   Example: `print_config("RandomForest", n_estimators=100, max_depth=5, random_state=42)`

Task 4

Create a function **`base_calculator(a, b, operation="sum")`**. Then, create a "wrapper" function called **`smart_wrapper(*args, **kwargs)`**. The `smart_wrapper` should:

1. Print "Logging: Function call initiated."
2. Pass all received arguments and keyword arguments directly into `base_calculator` using the **Unpacking (Splat) Operator**.
3. Return the result of the calculation.

Task 5

Implement a recursive function to navigate and "flatten" a deeply nested directory structure.

Scenario: In large-scale AI datasets, metadata is often stored in complex, non-uniform formats. You are provided with a variable named files that represents a file system. This structure is a mix of:

1. Dictionaries: Representing folders (Keys are folder names, Values are content).
2. Lists: Representing a collection of multiple sub-folders or file groups.
3. Strings: Representing the actual filenames (the "leaves" of our tree).

Task: Write a recursive function list_all_files(data) that traverses this entire structure. The function must:

- Identify if the current object is a Dictionary, a List, or a String.
- Recursively dive into dictionaries and lists until a string is found.
- Print the name of every directory and the names of files found within, regardless of its depth.
- Handle Empty Cases: Ensure the function does not crash if it encounters an empty list [] or an empty dictionary {}.

Example:

```
files = {
    "docs": [
        {"work": ["resume.pdf", "project.docx"]},
        {"personal": [
            {"pictures": ["photo.jpg", "vacation1.jpg", "vacation2.jpg"]},
            {"videos": []}
        ]},
        "todo.txt", "logfile.csv",
        {"downloads": []}
    ]
}

# Expected Output:
Directory: docs
Directory: work
  -> File: resume.pdf
  -> File: project.docx
Directory: personal
Directory: pictures
  -> File: photo.jpg
  -> File: vacation1.jpg
  -> File: vacation2.jpg
Directory: videos
  -> File: todo.txt
  -> File: logfile.csv
Directory: downloads
```